Guía paso a paso

Conectar Neon (PostgreSQL en la nube) con pgAdmin

Incluye cómo cargar el modelo de base de datos y cómo dejar la URL lista para tu archivo .env

Qué vas a lograr

- Una base de datos **neondb** en Neon, visible y administrable desde pgAdmin 4.
- El modelo de la aplicación de búsqueda (6 tablas) cargado y verificado.
- La cadena de conexión lista para usar en tu aplicación (archivo **.env**).

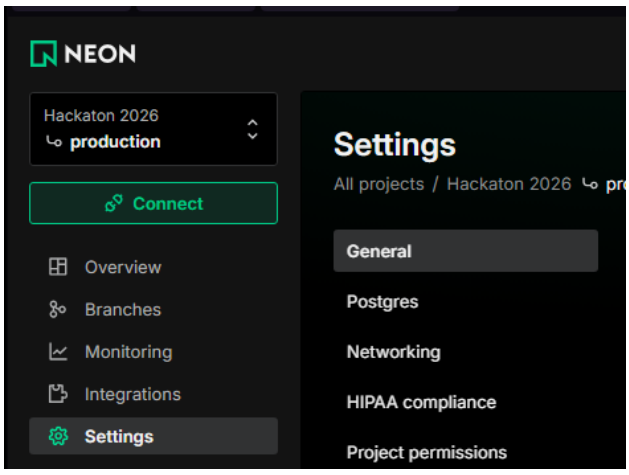
Antes de empezar

- Una cuenta en Neon con un proyecto creado (en esta guía: **Hackaton 2026**, rama **production**).
- pgAdmin 4 instalado en tu computadora.
- El archivo **schema_reniec_v3.txt** con el script de la base de datos.

Parte 1 · En Neon: obtener los datos de conexión

1 Abrir el proyecto y pulsar Connect

Entra a **console.neon.tech** e inicia sesión. Abre tu proyecto y, en el panel de la izquierda, pulsa el botón verde **Connect**.

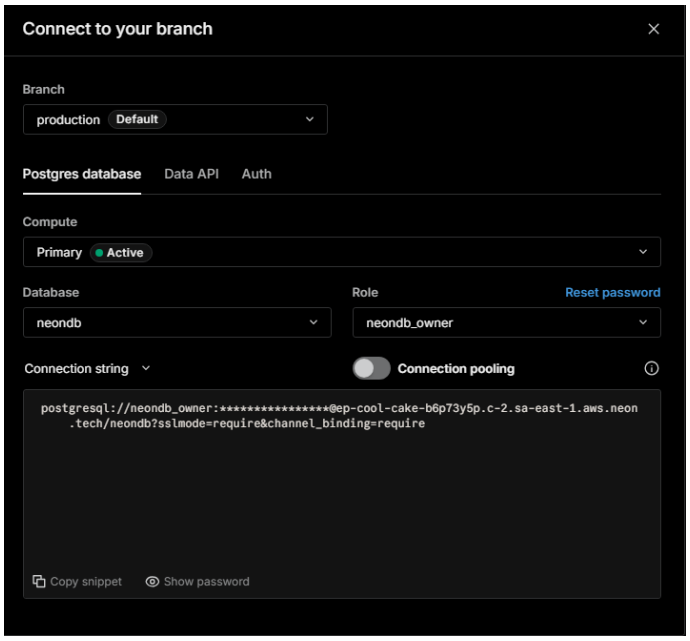


Panel de Neon: proyecto «Hackaton 2026», rama production y botón Connect.

2 Revisar la ventana «Connect to your branch»

Se abre una ventana con la cadena de conexión. Comprueba estos valores:

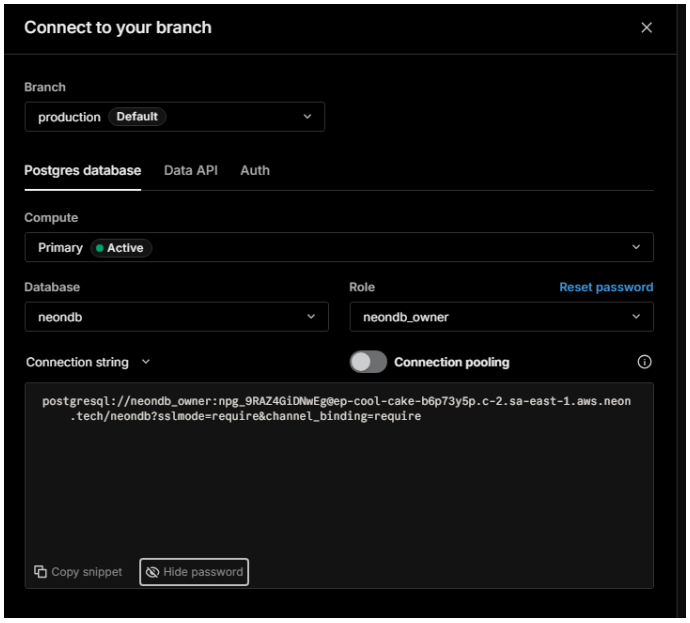
- **Branch:** production
- **Compute:** Primary (Active)
- **Database:** neondb
- **Role:** neondb_owner
- **Connection pooling: apagado**. Para pgAdmin se usa la cadena directa, sin `-pooler` en el host.



Con el pooling apagado, la cadena no incluye «-pooler».

3 Mostrar y copiar la contraseña

Pulsa **Show password**. La clave aparece dentro de la cadena, entre `neondb_owner:` y `@`. Cópiala: la necesitarás en pgAdmin.



Con «Show password» la clave se muestra dentro de la cadena.

> **Cuida tu contraseña.** No la compartas ni la subas a GitHub. Si alguien la llega a ver, pulsa **Reset password** en esta misma ventana para generar una nueva.

Cómo se lee la cadena de conexión

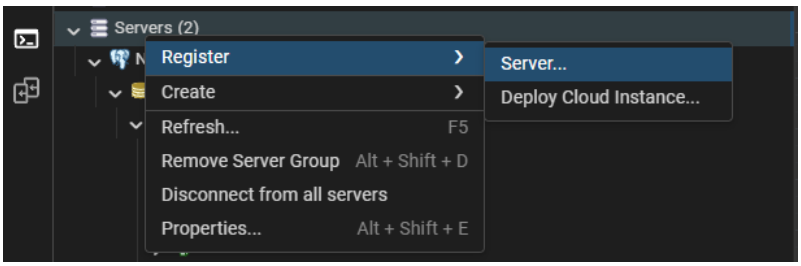
```
postgresql://ROL:CLAVE@HOST/BASE?sslmode=require&channel_binding=require
```

Parte de la cadena	Qué es	Campo en pgAdmin
ROL	El usuario de la base (aquí: neondb_owner)	Username
CLAVE	La contraseña del paso 3	Password
HOST	El servidor: lo que va entre @ y /	Host name/address
BASE	El nombre de la base (aquí: neondb)	Maintenance database
sslmode=require	Neon exige conexión cifrada (SSL)	Parameters > SSL mode

Parte 2 · En pgAdmin: registrar el servidor

4 Abrir el cuadro Register - Server

En el explorador de objetos (panel izquierdo), haz clic derecho sobre **Servers** y elige **Register > Server...**



Menú contextual de Servers en pgAdmin.

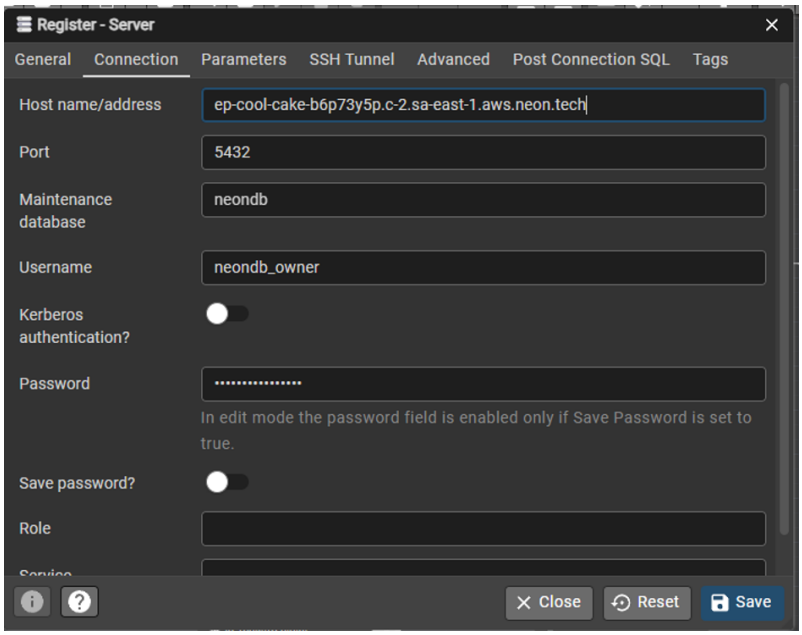
5 Pestaña General: ponerle nombre

En **General > Name** escribe el nombre que quieras para este servidor, por ejemplo **Neon hackaton**. Es solo una etiqueta dentro de pgAdmin.

6 Pestaña Connection: completar los datos

Completa los campos con los datos de Neon:

Campo	Valor
Host name/address	`ep-cool-cake-b6p73y5p.c-2.sa-east-1.aws.neon.tech`
Port	`5432`
Maintenance database	`neondb`
Username	`neondb_owner`
Password	`la clave que copiaste en el paso 3`

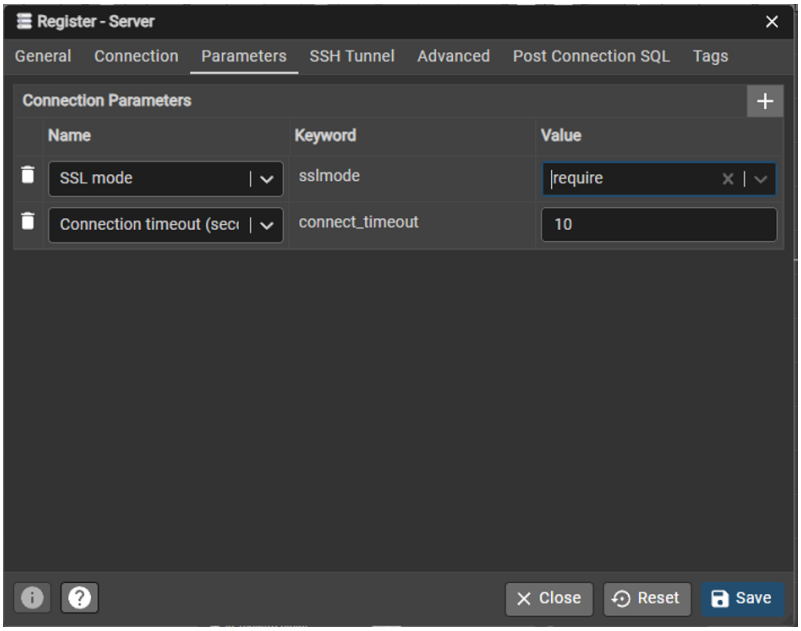


Pestaña Connection con el host, el puerto, la base y el usuario.

> En el campo **Host** escribe solo el servidor, sin `postgresql://`, sin la barra final y sin espacios. Tu host será distinto: usa el de tu propia cadena.

7 Pestaña Parameters: SSL mode = require

Abre la pestaña **Parameters** y comprueba que **SSL mode** sea **require**. Neon rechaza las conexiones sin cifrar. En versiones anteriores de pgAdmin esta opción está en una pestaña llamada **SSL**.

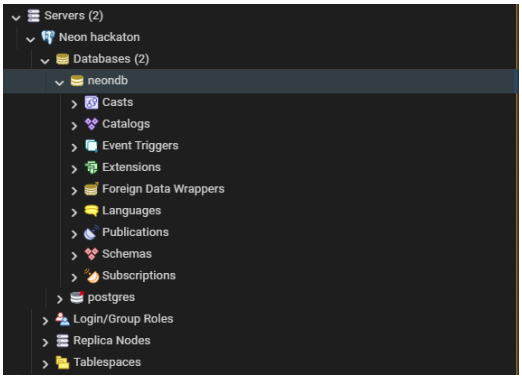


Parameters: SSL mode en require (el timeout de 10 segundos es el valor por defecto).

Pulsa **Save** para guardar el servidor.

8 Verificar que la conexión funciona

En el explorador debe aparecer el servidor con sus bases de datos. Despliega **Neon hackaton > Databases > neondb**: si se abre sin errores, ya estás conectado.



Servidor conectado: se ve la base neondb dentro de Neon.

Parte 3 · Cargar el modelo de la base de datos

9 Ejecutar el script del modelo

Haz clic derecho sobre **neondb** y elige **Query Tool**. Abre **schema_reniec_v3.txt**, copia **todo** su contenido, pégalo en el editor y presiona **F5**.

Después puedes cargar los datos de prueba con **datos_prueba_v3.txt**, ejecutándolo por bloques: selecciona un bloque y presiona F5.

10 Comprobar que se crearon las tablas

Ejecuta esta consulta en el Query Tool:

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;
```

Debes ver 6 tablas: `busqueda_dni`, `busqueda_ruc`, `inicio_sesion`, `registro`, `token_cambio_password` y `usuario`. También las verás en **neondb > Schemas > public > Tables** (clic derecho en Schemas > Refresh).

Parte 4 · Usar la conexión en tu aplicación (.env)

En el archivo **.env** de tu proyecto guarda la cadena completa en **una sola línea**, reemplazando `TU_CLAVE` por tu contraseña:

```
DATABASE_URL="postgresql://neondb_owner:TU_CLAVE@ep-cool-cake-b6p73y5p.c-2.sa-east-1.aw
s.neon.tech/neondb?sslmode=require&channel_binding=require"
```

- Agrega **.env** a tu **.gitignore** para no subirlo nunca a GitHub.
- Si la contraseña tiene símbolos como @, / o #, escríbelos codificados: @ = %40, / = %2F, # = %23.
- Para la aplicación en producción, Neon recomienda la cadena **pooled**: la misma, con `-pooler` después del id del host (activando Connection pooling en Neon).

Parte 5 · Si algo falla

Síntoma	Causa probable	Qué hacer
No encuentra el servidor («could not translate host name»)	Host mal copiado: espacios, saltos de línea o texto de más	Copia de nuevo solo el host: lo que va entre @ y /
«password authentication failed»	Clave mal copiada o cambiada con Reset password	Usa Show password en Neon y pégala de nuevo en la pestaña Connection
Error de SSL o conexión rechazada	SSL mode distinto de require	Parameters > SSL mode = require

Síntoma	Causa probable	Qué hacer
Tarda y termina en timeout	Sin internet, firewall o servidor iniciando	Espera unos segundos y vuelve a intentar; revisa tu red
«permission denied» al crear pgcrypto	Permisos del rol	Ejecuta solo CREATE EXTENSION IF NOT EXISTS pgcrypto; con el rol neondb_owner y repite el script. Si sigue, copia el mensaje completo
«relation already exists»	Ya ejecutaste el script antes	Borra lo anterior (solo en esta BD de pruebas) con el bloque de abajo y vuelve a ejecutar

Reiniciar la base de pruebas

```
BEGIN;
DROP FUNCTION IF EXISTS cambiar_password_con_token(TEXT, VARCHAR);
DROP FUNCTION IF EXISTS generar_token_cambio_password(VARCHAR, INT);
DROP TABLE IF EXISTS busqueda_ruc, busqueda_dni, usuario,
token_cambio_password, inicio_sesion, registro;
COMMIT;
```

Úsalo solo en tu base de pruebas. Borra las tablas y todos sus datos. Nunca lo ejecutes en la base de un cliente o de otro proyecto.